

ScanConnect × Gupshup

WhatsApp Business API Integration — Requirements Checklist

Why we need this

ScanConnect's QR tags let someone who scans a vehicle's sticker message the owner or an emergency contact without ever seeing their real phone number. Today that "Message" button opens a plain `wa.me` deep link straight to the owner's real WhatsApp number — a placeholder we put in so the flow works end-to-end while the real integration is pending. We want to replace it with Gupshup's WhatsApp Business API so messages are sent through our own WhatsApp Business number, with the vehicle owner's number never exposed to the person scanning the tag, and the owner able to see who's contacting them and why before replying.

Separately, ScanConnect also has a masked-calling flow (voice) still pending a telephony provider — that piece is being scoped independently (currently pointed at Knowlarity) and is **not** what this document is asking Gupshup to provide, though we've noted at the end where Gupshup's own voice/masking products might overlap, in case combining both into one vendor relationship is worth discussing.

Where we are today (for context)

So the requirements below make sense, here's what already exists on our side:

- The "Message" button, after the scanner verifies the last 4 digits of the vehicle's plate, opens `wa.me/<real number>?text=<reason>` directly in a new tab — this is the exact call site Gupshup's API would replace.
- We already collect a structured "reason for contact" (e.g. "The car is in no parking", "The car is getting towed") before the message is sent — this maps naturally to a WhatsApp message template's variable fields.
- We have one existing in-app notification system (a database row + bell icon in our dashboard) that fires when someone tries to contact an owner. No email or SMS sending exists yet. A Gupshup-sent WhatsApp message would be a new, additional channel alongside this.
- We have one existing pattern for a third-party server-to-server credential (a single shared API-key header, currently used for an unrelated telephony partner) that any Gupshup credentials would follow the same shape as.
- We do not yet have any webhook endpoint for inbound WhatsApp messages or delivery-status callbacks — that would be new.

What we need from the Gupshup team

1. Account & WhatsApp Business number setup

- A Gupshup Partner/Enterprise account provisioned for ScanConnect (or confirmation of which existing account to use, if one already exists).

- A dedicated WhatsApp Business phone number registered to ScanConnect's Meta Business Manager/WABA — confirmation of whether we provide a new number or migrate an existing one, and whether that number can currently receive an SMS/voice call for the one-time verification step.
- Confirmation of Meta Business Verification status/requirements for our account, and the messaging-volume tier (unverified numbers are capped at 250 business-initiated conversations per rolling 24 hours) — we need to know if our expected volume needs verification before launch.
- Display name approval for the WhatsApp Business profile (business name, logo, category) that will show to vehicle owners when they receive a message.

2. API credentials & environment details

- API key / access token for sending messages (production and, if available, a sandbox/staging key for our dev environment).
- App ID / Source (WABA) number ID that identifies our registered number in API calls.
- Base API URL / region endpoint we should send requests to.
- Confirmation of rate limits (messages per second, per day) for our tier, so we can size our queue/retry logic correctly.

3. Message template approval

WhatsApp requires pre-approved templates for any business-initiated message (i.e. any message that isn't a reply within an existing 24-hour customer-service window — which is our exact case, since the vehicle owner hasn't messaged us first). We need Gupshup's help submitting and getting these approved by Meta:

- A template for the initial contact notification, with variables for: the reason selected (e.g. "The car is in no parking"), the vehicle's masked plate number, and a way for the owner to respond/call back without the sender's number being exposed either.
- Guidance on Meta's template category rules (utility vs. marketing) so this qualifies as a utility message and isn't rate-limited or filtered as promotional.
- Expected turnaround time for template approval, so we can plan a launch date.

4. Webhook / callback setup

- The exact webhook payload shape Gupshup will POST to us for delivery status (sent/delivered/read/failed) and for inbound replies from the vehicle owner, so we can build a matching receiver endpoint.
- The authentication mechanism for that webhook (signature header, shared secret, IP allow-list, or similar) — we currently use HMAC signature verification for our one other signed webhook (payments) and would follow the same pattern if Gupshup supports it.
- Confirmation of what a public HTTPS endpoint needs to look like on our side (timeout limits, expected response codes, retry/backoff behavior on Gupshup's end if our endpoint is briefly down).

5. Compliance & opt-in

- Guidance on WhatsApp's opt-in requirements as they apply to us: the vehicle owner registers their own number with ScanConnect (not the person scanning the tag), so we need to confirm whether owner registration itself counts as sufficient opt-in for business-initiated utility messages, or if an explicit additional opt-in step is required.

- Any data-retention or data-residency requirements tied to the account's chosen storage region, since we store the owner's real phone number in our own database regardless of the messaging channel.

6. Cost & commercials

- Per-conversation / per-message pricing for utility-category templates in India (our primary market), and whether pricing differs for the masked-calling/voice product if we explore that later.
- Minimum commitment, billing cycle, and wallet/credit top-up mechanism.

Summary: assets we're asking Gupshup to provide

Item	Needed for
Provisioned account + registered WhatsApp Business number	Sending messages from our own brand, not a generic number
API key / access token (prod + sandbox)	Authenticating our server's API calls
WABA / source number ID	Identifying our number in every API request
Approved message template(s)	Legally sending the first, business-initiated message to an owner
Webhook payload spec + auth mechanism	Building our delivery-status and inbound-reply receiver
Rate limits for our tier	Sizing our send queue and retry logic
Pricing sheet + billing setup	Budgeting and finance sign-off before launch

Note: masked/virtual-number voice calling is a separate, independently-scoped piece of work (see "Why we need this" above). If Gupshup's own voice or number-masking product is a fit, we'd want that scoped as a distinct follow-up, not bundled into this WhatsApp messaging request.